

DECODELABS: DATA ANALYTICS INTERNSHIP

Project 3: SQL Data Analysis Report

Intern Name: Ravi kishan

Date Submitted: 16th July 2026

Database: DecodeLabs

Dataset: Cleaned Dataset for Data Analytics (DecodeLabs)

Table: orders

Total Rows: 1,200

Tool Used: PostgreSQL 18 / pgAdmin 4

QUERY 1: Sample Data

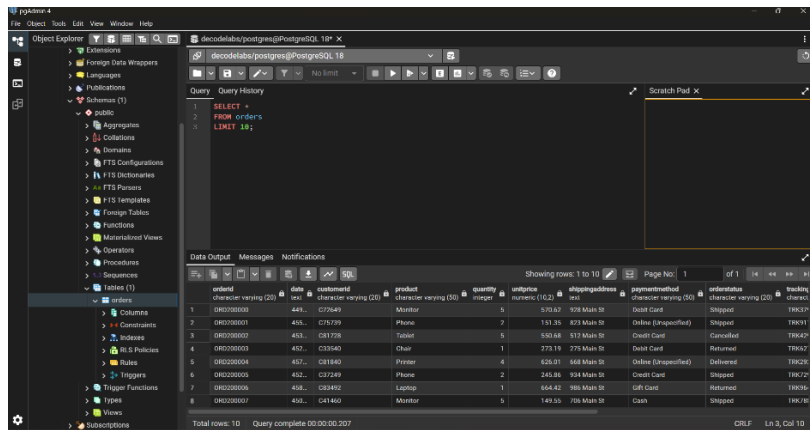
Business Question: What does our raw dataset look like?

The dataset contains 1,200 order records across 14 columns spanning January 2023 to June 2025.

SQL Code:

```
SELECT *  
FROM orders  
LIMIT 10;
```

Result:



orderid	date	customid	product	quantity	unitprice	shippingaddress	paymentmethod	orderstatus	tracking	trackingid
ORD000001	4451	C75491	Monitor	5	570.67	879 Main St	Debit Card	Shipped		TRK01
ORD000002	4451	C75799	Phone	2	131.35	879 Main St	Online (Unspecified)	Shipped		TRK01
ORD000003	4531	C81778	Tablet	5	550.68	879 Main St	Credit Card	Cancelled		TRK42
ORD000004	4531	C33040	Chair	1	273.19	276 Main St	Debit Card	Returned		TRK67
ORD000005	4531	C81840	Printer	4	626.01	668 Main St	Online (Unspecified)	Delivered		TRK25
ORD000006	4531	C37249	Phone	2	245.89	884 Main St	Credit Card	Shipped		TRK72
ORD000007	4531	C34362	Laptop	1	654.82	885 Main St	Gift Card	Returned		TRK76
ORD000008	4531	C41460	Monitor	5	149.55	706 Main St	Cash	Shipped		TRK78

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 2: Business KPIs

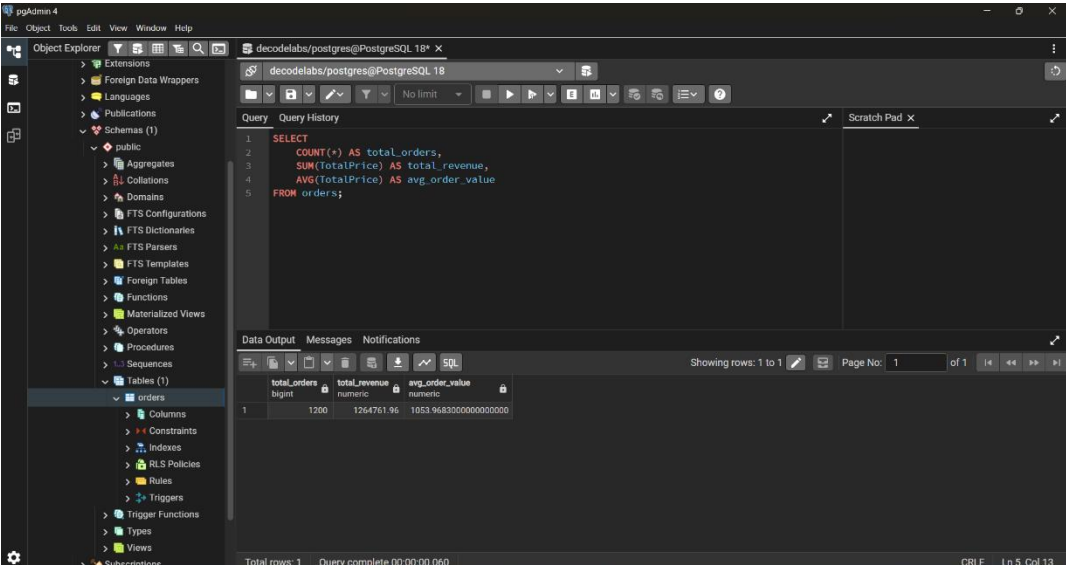
Business Question: What is our Total Revenue and Average Order Value?

Total revenue is \$1,264,761.96 across 1,200 orders with an average order value of \$1,053.97.

SQL Code:

```
SELECT
COUNT (*) AS total_orders,
SUM (TotalPrice) AS total_revenue,
AVG (TotalPrice) AS avg_order_value
FROM orders;
```

Result:



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure with 'orders' selected under 'Tables (1)'. The central pane shows the SQL query: `SELECT COUNT(*) AS total_orders, SUM(TotalPrice) AS total_revenue, AVG(TotalPrice) AS avg_order_value FROM orders;`. The bottom pane shows the query results in a table with 1 row and 3 columns: `total_orders` (bigint), `total_revenue` (numeric), and `avg_order_value` (numeric). The values are 1200, 1264761.96, and 1053.9683000000000000 respectively. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.060'.

total_orders	total_revenue	avg_order_value
1200	1264761.96	1053.9683000000000000

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 3: Revenue by Product

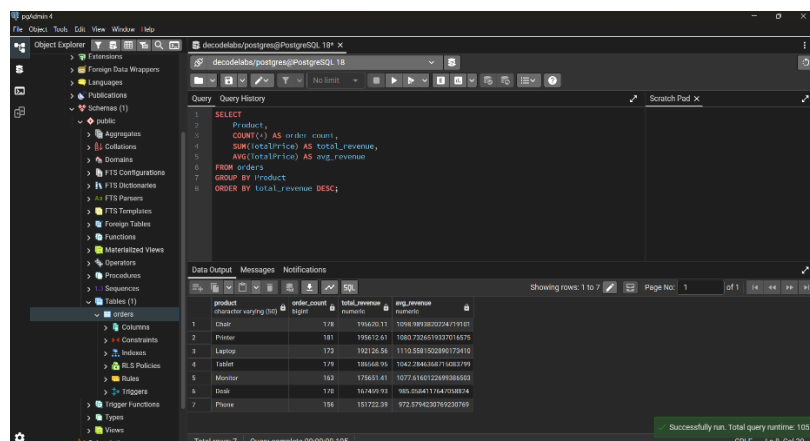
Business Question: What is our Top Revenue Product, Lowest Revenue Product and Highest Average Order Value Product?

Chair is the top revenue product at \$195,620 while Phone is the lowest at \$151,722. Laptop has the highest average order value at \$1,111.

SQL Code:

```
SELECT
Product,
COUNT(*) AS order_count,
SUM (TotalPrice) AS total_revenue,
AVG (TotalPrice) AS avg_revenue
FROM orders
GROUP BY Product
ORDER BY total_revenue DESC;
```

Result:



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
Product,
COUNT(*) AS order_count,
SUM (TotalPrice) AS total_revenue,
AVG (TotalPrice) AS avg_revenue
FROM orders
GROUP BY Product
ORDER BY total_revenue DESC;
```

The results are displayed in a table with the following columns: product, order_count, total_revenue, and avg_revenue. The data is sorted by total_revenue in descending order.

product	order_count	total_revenue	avg_revenue
Chair	178	195620.11	1098.98830224719101
Printer	181	195612.61	1080.732659337014475
Laptop	172	192126.56	1110.5581502290172410
Tablet	179	186048.99	1042.284866714082799
Monitor	162	178613.81	1077.8612259904802
Desk	170	161449.93	950.004117447038024
Phone	156	151722.39	972.5794230740230769

Successfully ran. Total query runtime: 105 ms

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 4: Cancelled Orders

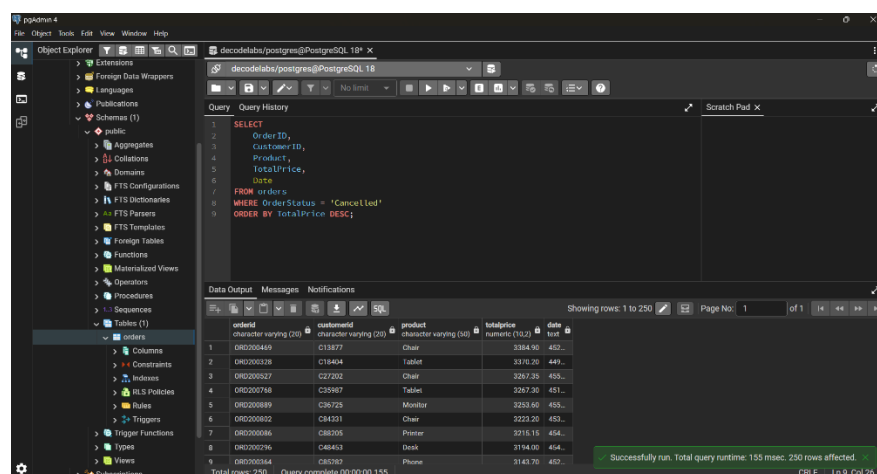
Business Question: How many cancelled orders represent lost Revenue?

250 orders were cancelled representing significant lost revenue. The highest single cancellation was \$3,384.90 for a Chair order.

SQL Code:

```
SELECT
OrderID,
Customer ID,
Product,
TotalPrice,
Date
FROM orders
WHERE OrderStatus = 'Cancelled'
ORDER BY TotalPrice DESC;
```

Result:



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including schemas, tables, and views. The central pane shows a SQL query that has been executed. The bottom pane displays the results of the query, which are 250 rows of data. The results are sorted by TotalPrice in descending order. The first row shows an OrderID of 080200449, a CustomerID of C13877, a Product of Chair, a TotalPrice of 3384.90, and a Date of 452-01-01. The last row shows an OrderID of 080200364, a CustomerID of C85769, a Product of Phone, a TotalPrice of 3143.70, and a Date of 452-01-01. A green status bar at the bottom indicates that the query was successfully run, with a total query runtime of 155 msec and 250 rows affected.

orderid	customerid	product	totalprice	date
080200449	C13877	Chair	3384.90	452-01-01
080200328	C18404	Tablet	3376.20	449-01-01
080200527	C27202	Chair	3267.35	453-01-01
080200768	C35887	Tablet	3267.30	451-01-01
080200889	C36725	Monitor	3253.60	453-01-01
080200402	C84231	Chair	3223.20	453-01-01
080200096	C86305	Pieter	3215.15	454-01-01
080200296	C48463	Desk	3194.00	454-01-01
080200364	C85769	Phone	3143.70	452-01-01

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 5: Order Status

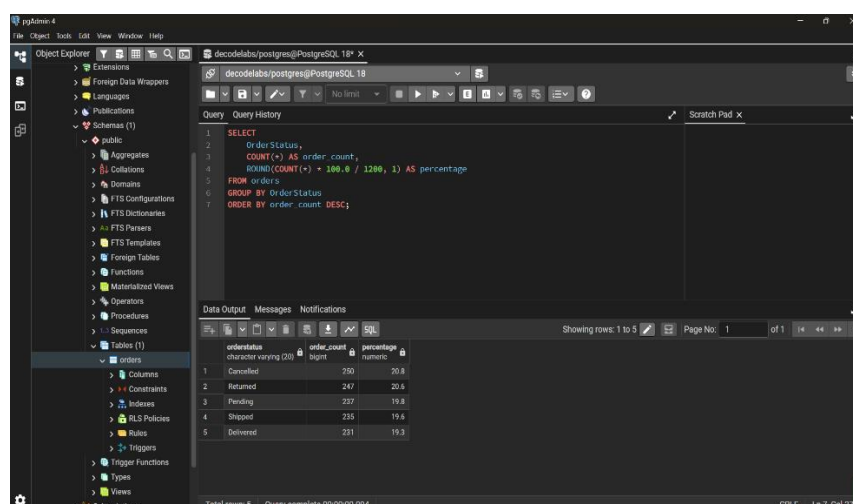
Business Question: What is the comprehensive Order Status?

Only 19.3% of orders are successfully Delivered. Combined Cancelled and Returned orders represent 41.4% of all orders; a critical operational problem.

SQL Code:

```
SELECT
OrderStatus,
COUNT (*) AS order_count,
ROUND (COUNT (*) * 100.0 / 1200, 1) AS percentage
FROM orders
GROUP BY OrderStatus
ORDER BY order_count DESC;
```

Result:



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'orders' table. The central pane shows the SQL query being executed. The bottom pane displays the query results in a table format.

orderstatus	order_count	percentage
Cancelled	290	20.8
Returned	247	20.5
Pending	237	19.8
Shipped	235	19.6
Delivered	211	19.3

Total rows: 5 Query complete 00:00:00.094 ORF Ln 7, Col 27

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 6: VIP Orders

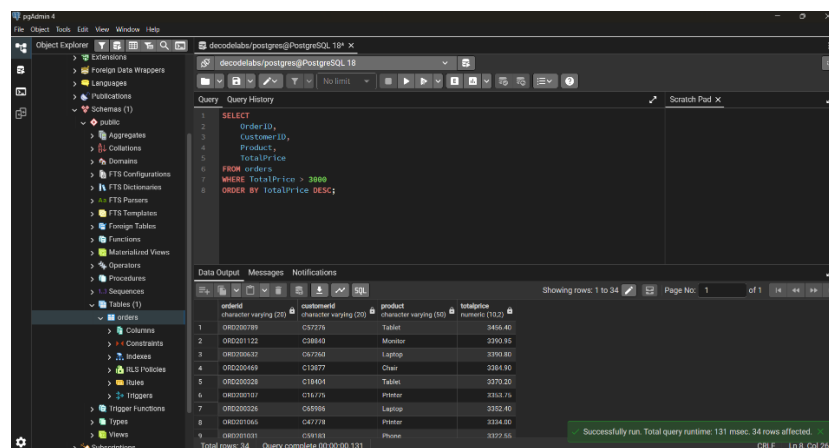
Business Question: How many VIP Orders in Total?

34 VIP customers placed orders above \$3,000. The highest single order was \$3,456.40 for a Tablet. These customers should be targeted with a loyalty program.

SQL Code:

```
SELECT
OrderID,
Customer ID,
Product,
TotalPrice
FROM orders
WHERE TotalPrice > 3000
ORDER BY TotalPrice DESC;
```

Result:



The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Shows the database structure including Schemas, Tables, Views, and Functions.
- Query Editor:** Displays the SQL query:

```
SELECT
OrderID,
CustomerID,
Product,
TotalPrice
FROM orders
WHERE TotalPrice > 3000
ORDER BY TotalPrice DESC;
```
- Data Output:** Shows the results of the query in a table format. The table has 4 columns: orderid, customerid, product, and totalprice. The results are sorted by totalprice in descending order.
- Status Bar:** Indicates "Successfully run. Total query runtime: 131 msec. 34 rows affected."

orderid	customerid	product	totalprice
ORD000789	C37279	Tablet	3456.40
ORD001122	C38849	Monitor	2990.95
ORD000632	C67763	Laptop	2990.80
ORD000469	C12877	Chair	2184.90
ORD000228	C18404	Tablet	2070.20
ORD001767	C16775	Printer	2063.75
ORD000268	C43963	Laptop	2052.40
ORD001065	C47778	Printer	2034.90
ORD001631	C16181	Phone	2022.65

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 7: Referral Source

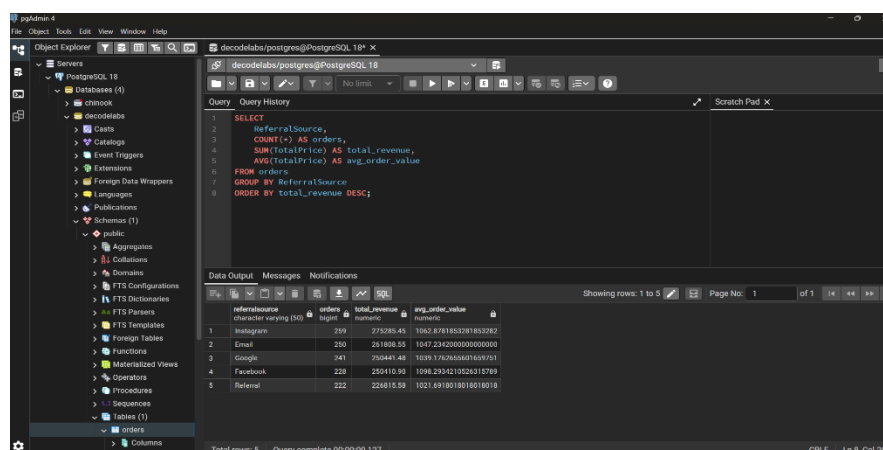
Business Question: How has the referral program performed?

Instagram is the top marketing channel generating \$275,285 from 259 orders. The Referral program is the weakest at \$226,816 and needs restructuring.

SQL Code:

```
SELECT
ReferralSource,
COUNT (*) AS orders,
SUM (TotalPrice) AS total_revenue,
AVG (TotalPrice) AS avg_order_value
FROM orders
GROUP BY ReferralSource
ORDER BY total_revenue DESC;
```

Result:



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'decoodelabs'. The central pane shows the SQL query: `SELECT ReferralSource, COUNT(*) AS orders, SUM(TotalPrice) AS total_revenue, AVG(TotalPrice) AS avg_order_value FROM orders GROUP BY ReferralSource ORDER BY total_revenue DESC;`. The bottom pane displays the query results in a table with 5 rows and 4 columns: referralsource, orders, total_revenue, and avg_order_value.

referralsource	orders	total_revenue	avg_order_value
Instagram	259	275285.45	1063.27618523781832162
Email	250	261808.55	1047.24320000000000000
Google	341	350441.48	1029.1762653681609751
Facebook	228	250110.90	1096.2934210526315789
Referral	222	226815.58	1021.6910018018018018

Total rows: 5 Query complete 00:00:00.127

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 8: Monthly Trend

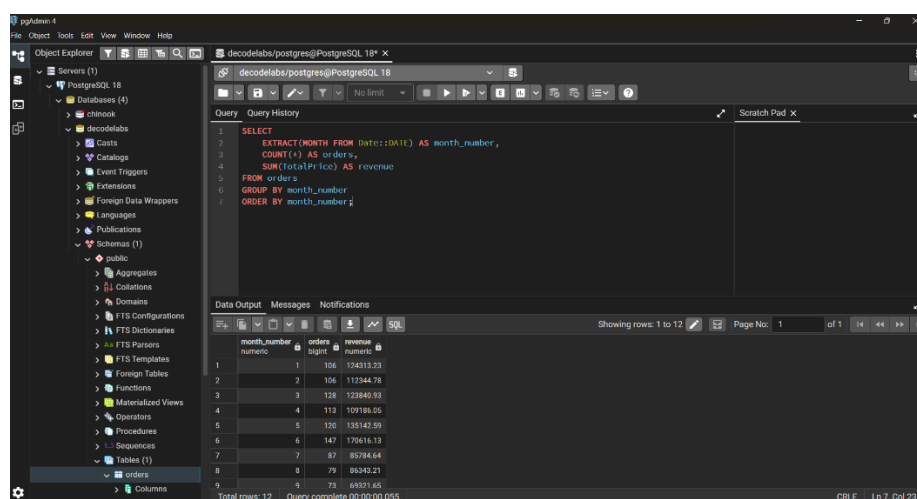
Business Question: What is the Highest and Lowest performing month?

June is the peak month at \$170,616 with 147 orders. September is the worst at \$69,322 with only 73 orders; a 59% revenue gap between best and worst months.

SQL Code:

```
SELECT  
EXTRACT (MONTH FROM Date::DATE) AS month_number,  
COUNT (*) AS orders,  
SUM (TotalPrice) AS revenue  
FROM orders  
GROUP BY month_number  
ORDER BY month_number;
```

Result:



The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 database. The query is executed, and the results are displayed in a table with 12 rows. The table has three columns: month_number, orders, and revenue. The data shows that June (month 6) has the highest revenue and number of orders, while September (month 9) has the lowest.

month_number	orders	revenue
1	106	124313.23
2	106	112344.78
3	128	123840.93
4	113	109186.05
5	120	135142.99
6	147	170616.13
7	87	85784.64
8	79	86342.21
9	73	69322.00

Total rows: 12 Query complete 00:00:00.055

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 9: Payment Method

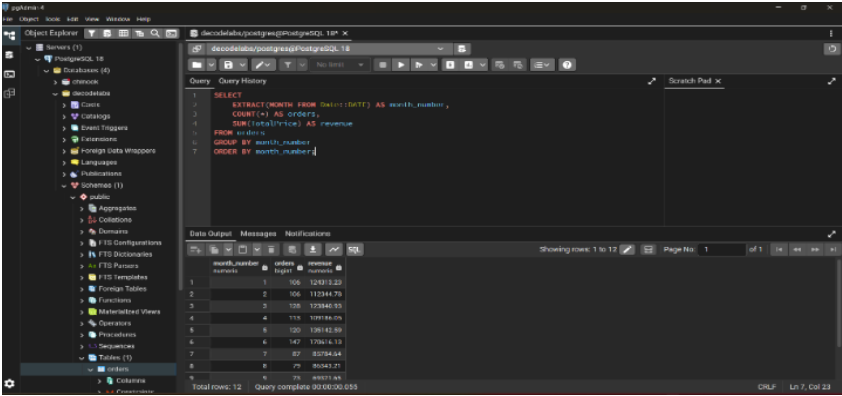
Business Question: What payment method generated the most revenue?

Credit Card generates the most revenue at \$263,848. Debit Card has the lowest average order value at \$1,002 suggesting smaller purchase behavior.

SQL Code:

```
SELECT  
PaymentMethod,  
COUNT (*) AS orders,  
SUM (TotalPrice) AS total_revenue,  
AVG (TotalPrice) AS avg_order_value  
FROM orders  
GROUP BY PaymentMethod  
ORDER BY total_revenue DESC;
```

Result:



The screenshot shows a SQL Server Enterprise Manager interface. The 'Query' tab is active, displaying the following SQL query:

```
SELECT  
    EXTRACT(MONTH FROM SalesDate) AS month_number,  
    COUNT(*) AS orders,  
    SUM(TotalPrice) AS revenue  
FROM orders  
GROUP BY month_number  
ORDER BY month_number
```

The 'Data Output' tab shows the results of the query, which are sorted by month_number. The results are as follows:

month_number	orders	revenue
1	106	12513.22
2	106	112344.75
3	108	112344.55
4	114	101144.75
5	100	101144.55
6	147	17614.13
7	87	8714.54
8	79	8714.54
9	79	8714.54

The status bar at the bottom indicates 'Total rows: 12' and 'Query complete 00:00:00.055'.

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.

QUERY 10: Coupon Analysis

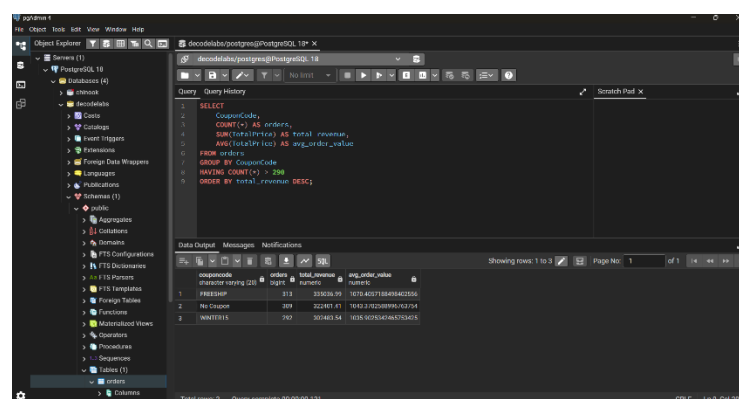
Business Question: Which Coupon was most used?

FREESHIP is the most used coupon with 313 uses generating \$335,037. Customers with No Coupon still generate \$322,401 showing strong organic demand.

SQL Code:

```
SELECT
CouponCode,
COUNT (*) AS orders,
SUM(TotalPrice) AS total_revenue,
AVG (TotalPrice) AS avg_order_value
FROM orders
GROUP BY CouponCode
HAVING COUNT (*) > 290
ORDER BY total revenue DESC;
```

Result:



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
  CouponCode,
  COUNT(*) AS orders,
  SUM(TotalPrice) AS total_revenue,
  AVG(TotalPrice) AS avg_order_value
FROM orders
GROUP BY CouponCode
HAVING COUNT(*) > 290
ORDER BY total_revenue DESC;
```

The results are displayed in a table with the following columns: couponcode, orders, total_revenue, and avg_order_value. The results are sorted by total_revenue in descending order.

couponcode	orders	total_revenue	avg_order_value
FREESHIP	313	335037.09	1069.14771849462556
No Coupon	289	322401.61	1113.84638968162754
WINTER15	797	301483.54	1015.90234144573425

Total rows: 3. Query complete 00:00:06.121. SQL - Ln 9, Col 29

Business Insight:

The dataset contains 1,200 order records across 14 columns including OrderID, Product, TotalPrice and OrderStatus.
